

Progetto: Spiegazione Della Mia Idea

Il mio progetto. No Spoiler Please! A TinyBERT Model to Detect Spoiler Reviews.

Scelte implementative: modelli scelti, hardware a disposizione e struttura della directory di lavoro

Valutazione finale e grafici

Metriche di performance (competenza)

Metriche di interpretabilità / reasoning quality (da valutare per la mia idea perchè uso un modello encoder che non è pensato per questo)

Metriche di efficienza e risorse (trade-off)

Cosa è già stato fatto

Come sfrutto la conoscenza dei primi 3 paper

Grafici ottenuti per il momento

Il mio progetto. No Spoiler Please! A TinyBERT Model to Detect Spoiler Reviews.

PREMESSA IMPORTANTE: QUESTA SEZIONE INCLUDE UNA MIA SPIEGAZIONE SCRITTA DI COSA VOGLIO FARE, I DETTAGLI MATEMATICI VERRANNO INCLUSI NELLE SEZIONI SUCCESSIVE RIGUARDANTI I PAPER ALLEGATI

Quante volte ci è capitato di leggere in giro la recensione di un film che non abbiamo ancora visto, ma siamo comunque curiosi di sapere il parere di chi l'ha già visto. In queste situazioni non è difficile incappare in recensioni che, fatto apposta per spoilerare o meno, includono spoiler riguardanti il film stesso (ad esempio il personaggio X muore o fa questa cosa, ecc..).

IMDb è in assoluto la fonte globale più famosa per film e serie TV e prevede anche recensioni lasciate dagli utenti sugli stessi. Immaginiamoci questa situazione: apro la recensione di un film che voglio vedere, la leggo e scopro che il personaggio X alla fine del film muore o fa questo e quello... (o, in generale, la recensione spolia qualcosa). Chiaramente voglio evitare una situazione del genere, ma finchè non leggo la recensione di un film non so effettivamente cosa ci sia scritto. E se esistesse un'estensione browser che, dato il testo di una recensione, mi dice chiaramente "spoiler!!" Oppure "no

spoiler"? Con l'evoluzione degli attuali LLM, questo task di classificazione binaria è realizzabile da un modello come BERT: per prima cosa si prende un grosso modello pre-addestrato su un corpus generico, ad esempio `bert-base-uncased` un grosso modello BERT con circa 110Mln di parametri pre-addestrato su grossi corpus di Wikipedia e BookCorpus, dopodichè si esegue fine tuning aggiungendo un livello di classificazione binaria in testa al modello e addestrandolo su un dataset contenente esempi di recensioni etichettate come spoiler/no spoiler. A tale scopo, il dataset individuato su Kaggle è `IMDB Spoiler Dataset`, che contiene un json chiamato `IMDB_reviews.json`. Questo enorme dataset di circa 900MB contiene circa 500k recensioni di utenti su IMDb ed ogni record contiene le seguenti informazioni:

- `review_date` : data della recensione
- `movie_id` : ID univoco del film interessato nella recensione
- `user_id` : ID univoco dell'utente recensore
- `is_spoiler` : etichetta binaria True/False che dice se la recensione è spoiler o meno
- `review_text` : testo della recensione
- `rating` : valutazione data dall'utente al film interessato
- `review_summary` : riassunto breve della recensione

Alla fine di questo fine tuning si vuole ottenere un grosso modello BERT specializzato nell'identificare le recensioni spoiler. Abbiamo però un problema: le estensioni girerebbero in locale sui PC degli utenti, e un modello grosso è sì molto esperto ma ha anche un costo computazionale decisamente grande, sia in termini di spazio occupato in memoria che di tempo di inferenza per produrre la previsione. Potremmo quindi usare e specializzare un modello più piccolo, che avrà quindi dei costi computazionali inferiori ma anche delle prestazioni proporzionalmente inferiori rispetto al modello grosso.

A questo scopo nasce la knowledge distillation, un concetto ideato da Hinton e poi raffinato da Jiao et al mediante un protocollo che costruisce un modello chiamato TinyBERT. L'idea è quella di prendere un grosso modello, detto teacher, e un modello più piccolo, detto student, e "trasferire" la conoscenza del modello grosso in quello più piccolo. In questo modo è stato dimostrato che si ottiene un modello più piccolo e compatto del teacher, ma con delle prestazioni molto vicine a quelle del teacher, rendendolo ideale per applicazioni in cui si deve curare il costo computazionale (nel mio caso, un'estensione che

deve dire all'utente OCCHIO CHE QUESTA RECENSIONE SPOILERA). Per fare questo, il protocollo di distillazione in due fasi di Jiao et al prevede:

1. Nella fase 1: General Distillation. Poichè il modello TinyBERT usato dagli autori è costruito da zero, prima di ricevere la specializzazione nel task dal teacher dovrebbe acquisire, sempre, dal teacher, della conoscenza generica. Per farlo si prende un grosso corpus (gli autori hanno preso wikipedia), lo si dà in pasto al teacher e si distilla allo student le loss di Hidden States, Embedding Layers e Attention Layers, ma non i logits (Output Layers).
 - a. Questa fase ho deciso di saltarla per i motivi spiegati in precedenza.
2. Nella fase 2: Task-Specific Distillation. Dopo aver trasferito allo student la conoscenza generale, si passa a specializzarlo sul task. Per prima cosa si fa fine tuning del teacher sul task (nel mio caso riconoscere una recensione come spoiler o no), prendendo il dataset task specific e (se necessario, ma penso di sì) aggiungendo un livello di classificazione binaria in cima al modello (se si dice così in gergo tecnico, correggimi se sbaglio). Dopodichè si esegue data augmentation del dataset (fondamentale se si hanno pochi dati, ma nel mio caso non è necessario perchè ho già una marea di dati), aggiungendo in corso di distillation anche una colonna con le predizioni del teacher per ogni esempio, lo si dà in pasto al teacher e nel frattempo si distilla allo student tramite loss di Hidden States, Embedding Layers, Attentions Layers e, a differenza della fase 1, anche dei logits (Output Layers). Per fare questo si distilla in due fasi distinte, descritte in seguito.

Alla fine di questo protocollo, si eseguono test e benchmark per confrontare le prestazioni dei due modelli, che presento nella sottosezione dedicata. Lo scopo di questo progetto, chiaramente, non è realizzare un'estensione per browser vera e propria, ma piuttosto verificare l'efficacia della KD in un task di classificazione binaria, con l'obiettivo di ottenere un modello performante come il teacher in questo task ma meno pesante.

Scelte implementative: modelli scelti, hardware a disposizione e struttura della directory di lavoro

Per quanto riguarda i modelli teacher e student scelti:

- Modello Teacher: bert-base-uncased. Si potrebbe usare bert-large-uncased, ma durante il fine tuning si è dimostrato essere più difficile da

addestrare e alla fine il delta di prestazioni non giustifica usare un modello con quasi il triplo dei parametri e più tempo di inferenza.

- Modello Student: huawei-noah/TinyBERT_General_4L_312D. Salto quindi la General Distillation perché **uso già uno student 'general TinyBERT'** (huawei-noah/TinyBERT_General_4L_312D), cioè già distillato sul dominio generale; quindi parto direttamente dalla task-specific distillation.

Userò un PC Windows 11 equipaggiato con una RTX 3070 da 8GB VRAM e 32GB di RAM.

Il codice va scritto in questo modo: dei moduli .py che implementano la logica delle singole funzionalità (download dati, pre processing dati, training ecc...) e poi notebook .ipynb che utilizzano questi moduli per realizzare concretamente le funzionalità + produzione di grafici e metriche benchmark

Per quanto riguarda la directory di lavoro, è organizzata nel seguente modo e va rispettata per non avere problemi. Le cartelle che ti elenco sono tutte dentro una cartella principale, non importa sapere come si chiama.

- Cartella `src` : contiene tutti i moduli .py
- Cartella `notebooks` : contiene tutti i notebook .ipynb
- Cartella `data` : contiene i dati utilizzati per l'addestramento. Potremmo realizzare due sottocartelle al suo interno
 - `data/raw` : dati "grezzi" scaricati ad esempio da kaggle
 - `data/processed` : dati elaborati (ad esempio post pre processing e split in train validation e test) e pronti per essere trasferiti ai modelli
- Cartella `checkpoints` : contiene i checkpoint dei modelli dopo il training, che serviranno come punto di partenza per le varie fasi della distillazione (ad esempio student post fase 1, teacher post fine tuning, post fase 2 e student post fase 2) in due fasi e per i benchmark dei due modelli.

Importante: laddove necessario ho messo un kaggle.json contenente le mie credenziali (username e key) per autenticarmi a Kaggle e poter scaricare il dataset. Questo .json verrà poi escluso con un .gitignore per non caricarlo nel mio repo github. Per questo motivo, il codice contenente le righe che scaricano il dataset Kaggle ha dovuto rispettare questa scelta implementativa, ma per le prossime fasi non ci serve

Valutazione finale e grafici

Per quanto riguarda i grafici richiesti rientrano in 3 categorie. Ti riporto i suggerimenti dati dal prof.

Metriche di performance (competenza)

Poiché la metodologia richiede di selezionare un **task specifico** (es. operazioni matematiche, riconoscimento di entità, classificazione) e non la conoscenza generale, le prime metriche devono misurare quanto bene lo studente esegue quel compito rispetto al maestro.

- **Metriche Quantitative (Accuracy, F1-Score, Precision/Recall):** A seconda del task scelto (classificazione o generazione), dovrai calcolare le metriche standard. Se il task è di classificazione (es. "Questo testo parla di sport o politica?"), userai **Accuracy** e **F1-Score**.
- **Misura del "Performance Drop" (Calo di Prestazioni):** È stato esplicitamente richiesto di quantificare l'eventuale calo di performance. Dovrai calcolare il delta tra lo score del Teacher e quello dello Student. L'obiettivo è minimizzare questo gap.

Grafici Suggeriti:

- **Bar Chart Comparativo:** Un grafico a barre raggruppate che mostri le performance (es. Accuracy) del Teacher accanto a quelle dello Student sul test set.
- **Matrice di Confusione Comparata:** Se il task è di classificazione, mostra due matrici di confusione affiancate: una per il Teacher e una per lo Student. Questo aiuta a capire se lo Student commette gli *stessi tipi di errori* del Teacher o se ne introduce di nuovi.

Metriche di interpretabilità / reasoning quality (da valutare per la mia idea perchè uso un modello encoder che non è pensato per questo)

Una parte specifica della metodologia suggerita prevede che il Teacher generi non solo le predizioni, ma anche le **catene di ragionamento** (*reasoning chains*) che lo studente deve apprendere.

- **Similarità del Ragionamento:** Se il dataset sintetico include il "perché" di una risposta (es. passaggi logici), dovresti valutare se lo Student ha imparato a ragionare come il Teacher, non solo a dare la risposta finale.

- **Metriche:** Potresti usare metriche di similarità testuale (come la cosine similarity tra gli embeddings delle spiegazioni generate) o metriche come **BLEU/ROUGE** se confronti il testo generato dallo Student con quello del Teacher.

Grafici Suggesti: Istogramma della Similarità: Un grafico che mostra la distribuzione dei punteggi di similarità tra le spiegazioni generate dal Teacher e dallo Student.

Come per ogni progetto di deep learning, è necessario mostrare che l'apprendimento è avvenuto correttamente.

- **Funzione di Loss (Distillation Loss):** Nella Knowledge Distillation, la loss function spesso combina la loss rispetto alle etichette reali e la loss rispetto alle probabilità (logits) del Teacher (la cosiddetta *distillation loss*).

Grafici Suggesti: Curve di Loss (Training & Validation): Un grafico lineare che mostra l'andamento della loss durante le epoche di addestramento dello Student. È fondamentale per dimostrare che il modello ha converso e non è andato in overfitting.

È stato menzionato che un vantaggio dei modelli più piccoli è la loro maggiore **interpretabilità**.

- **Analisi Qualitativa:** Sebbene non sia un grafico quantitativo, riportare esempi specifici (tabella) dove:
 1. Il Teacher risponde correttamente.
 2. Lo Student risponde correttamente (imitando il Teacher).
 3. Casi in cui lo Student fallisce dove il Teacher riesce.
- **Visualizzazione dell'Attenzione (Opzionale):** Se si vuole "hackerare" il modello, si potrebbe visualizzare se lo Student "guarda" (tramite meccanismi di attenzione) le stesse parole chiave del Teacher per prendere una decisione. Penso che si riferisca al plottare le heatmap delle matrici di attenzione e le matrici nascoste di teacher e student.

Metriche di efficienza e risorse (trade-off)

La motivazione principale della Knowledge Distillation è ecologica e legata alle risorse: i modelli grandi sono costosi e lenti. Pertanto, è cruciale dimostrare il vantaggio in termini di efficienza.

- **Dimensione del Modello (Parameter Count):** Confronta il numero di parametri del Teacher con quello dello Student.
- **Tempo di Inferenza (Inference Time / Latency):** Misura quanto tempo impiegano i due modelli a processare lo stesso set di dati.

Grafici Suggeriti: Grafico a Dispersione (Scatter Plot) Performance vs. Size:

Sull'asse X metti la dimensione del modello (o il tempo di inferenza) e sull'asse Y la performance (Accuracy). L'obiettivo è mostrare che lo Student si trova in un punto "ottimale" (alta accuracy, basse risorse) rispetto al Teacher.

Cosa è già stato fatto

Primo problema affrontato: dimensioni dataset. Ho deciso di creare un campione casuale di 300k elementi totali, diviso poi in train (80%), val e test set (10% ciascuno). Lo split è stato fatto grouped by movie_id per evitare che uno stesso film finisse in set diversi e la rete imparasse pattern del tipo "se vedi una qualunque recensione di questo film, è/non è sicuramente uno spoiler). Inoltre il dataset è sbilanciato sulle etichette (anche il campione di 300k elementi lo è), dato che circa il 76% degli esempi sono NO SPOILER. Per questo motivo ho usato una weighted cross entropy: peso di più le loss su errori su spoiler mentre le loss su errori su no spoiler hanno peso = 1 (come se non ci fosse quindi). Il peso è stato calcolato come rapporto tra etichette NO ed etichette SI sul train set, ma si aggira quasi sempre attorno a 2,8.

Il quarto paper presenta una strategia efficace specifica per il dataset IMDB spoiler detection: visto che il limite di token massimi in BERT large è 512 e la lunghezza delle recensioni spesso rischia di sfiorare questo limite, si procede con una strategia head+tail:

- Per prima cosa si tokenizza il testo della review e si controlla la lunghezza risultante
 - Se NON sfiora i 510 token, semplicemente si attacca all'inizio il token speciale [CLS] e alla fine [SEP]. Si aggiunge eventualmente del padding se non si raggiungono i 512 token finali.
 - Altrimenti
 - Si prendono i primi 128 token del testo della recensione e li si attaccano dopo il token [CLS] iniziale
 - Si attaccano poi gli ultimi 382 token del testo della recensione

- Si attacca infine il token speciale [SEP]: alla fine, si ottiene una sequenza di 512 token gestibile da BERT

In quel paper hanno dimostrato che spesso la parte centrale della recensione non contiene spoiler, e che quindi si può procedere in questo modo anzichè far gestire questa criticità al tokenizer di BERT large. Ho provato anche ad implementare una strategia a sliding window ma si è rivelata meno efficace di quella con head+tail.

Inoltre, viene presentata una strategia di learning rate decay in base ai layer della rete (assegnando un LR più basso al layer più basso per poi aumentarlo man mano che si sale), per prevenire catastrophic forgetting ed overfitting. LR base = $2e-5$ mentre il fattore di decadimento è 0.95; ho poi applicato un weight decay classico del deep learning di 0.01.

Il fine tuning è stato fatto per 3 epochs, usando come metrica di scelta del modello migliore la curva PR-AUC e ha avuto come risultati

- Training loss: 0.429
- Validation loss: 0.589
- Accuracy: 0.784
- F1: 0.596
- Precision: 0.58
- Recall: 0.614
- ROC-AUC: 0.813
- PR-AUC: 0.647

Ho poi calcolato, alla fine del fine tuning, 0.52 come threshold sul validation set, con la quale ho ottenuto sul test set:

- TEST F1: 0.5963893160202745
- TEST Precision: 0.5557206537890045
- TEST Recall: 0.6434803981811479
- TEST ROC-AUC: 0.8180763619593087
- TEST PR-AUC: 0.6563545279327828

Come sfrutto la conoscenza dei primi 3 paper

Questa sezione comprende maggiormente i dettagli matematici presentati nei paper e come questi vengono sfruttati nel mio progetto.

Il paper di Jiao et al, per la fase 2 di Task-Specific Distillation, decidono di procedere con la distillation in due sottofasi.

Introduco un minimo di notazione utile in seguito:

- $z_T(x)$ = Logit Teacher
- $z_S(x)$ = Logit Student
- $y_g \in \{0, 1\}$ Ground Truth
- $\sigma(\cdot)$ = Sigmoid function
- T = Temperatura KD
- $\rho \in [0, 1]$ = Soft ratio

Fase TD-1: (Intermediate / Transformer Distillation)

Dato un input x , la loss in forma compatta è

$$L_{TD1}(x) = \lambda_0 L_{embd}(x) + \sum_{m=1}^M \lambda_m (L_{hidn}^{(m)}(x) + L_{attn}^{(m)}(x))$$

dove

- Embedding distillation. Qui E_S = embedding student, E_T = embedding teacher e W_e = proiezione lineare per matchare le 312 dimensioni dello student sulle 768 del teacher.

$$L_{embd} = MSE(E_S W_e, E_T)$$

- Hidden-state distillation. Qui H_S = hidden states student, H_T = hidden states teacher e W_h = proiezione lineare.

$$L_{hidn} = MSE(H_S W_h, H_T)$$

- Attention distillation. Qui le attention vanno prese pre-softmax, dato che le librerie di HuggingFace solitamente restituiscono le attention post-softmax:

un modo semplice di farlo sarebbe la creazione di una piccola subclass dei moduli Attention (es. `BertSelfAttention`) e nel forward salvare anche `attention_scores` **prima della softmax** (quelli prima di `Softmax(dim=-1)`).

$$L_{attn} = \frac{1}{h} \sum_{i=1}^h MSE(A_i^S, A_i^T)$$

- Il classification head, in questa fase, viene congelato.

Si gestisce la corrispondenza layer student $\rightarrow$ layer teacher mediante mapping lineare $g(m)$: uso uno student con 4 layer quindi mappo a gruppi di 3 layer del teacher sullo student.

Per quanto riguarda gli altri parametri per questa fase:

- 8 epochs
- Batch size: 16 + gradient accumulation per arrivare ad un batch effettivo di 32
- Learning rate: evito LLRD, uso $5e-5$
- Optimizer/regolarizzazione: AdamW con $\beta_1 = 0.9$, $\beta_2 = 0.999$, dropout default di BERT e gradient clipping di 1.0
- Pesi loss:
 - $\lambda_{emd} = 1$
 - $\lambda_{hidn} = 1$
 - $\lambda_{attn} = 1.0$ se domina troppo, la abbasso.

Fase TD-2: Prediction-layer distillation

Qui si integra l'idea di Ji et al: si usano le hard labels per correggere le predizioni errate fatte dal prof, tenendo comunque in mente l'idea della weighted cross entropy loss applicata durante il fine tuning.

La logits loss diventa

$$L_{TD-2} = \rho(x) \cdot \underbrace{CE(z_T/T, z_S/T)}_{\text{Soft Loss (Non Pesata)}} \cdot T^2 + (1 - \rho(x)) \cdot \underbrace{WCE(z_S, y_{true})}_{\text{Hard Loss (PESATA)}}$$

Dove:

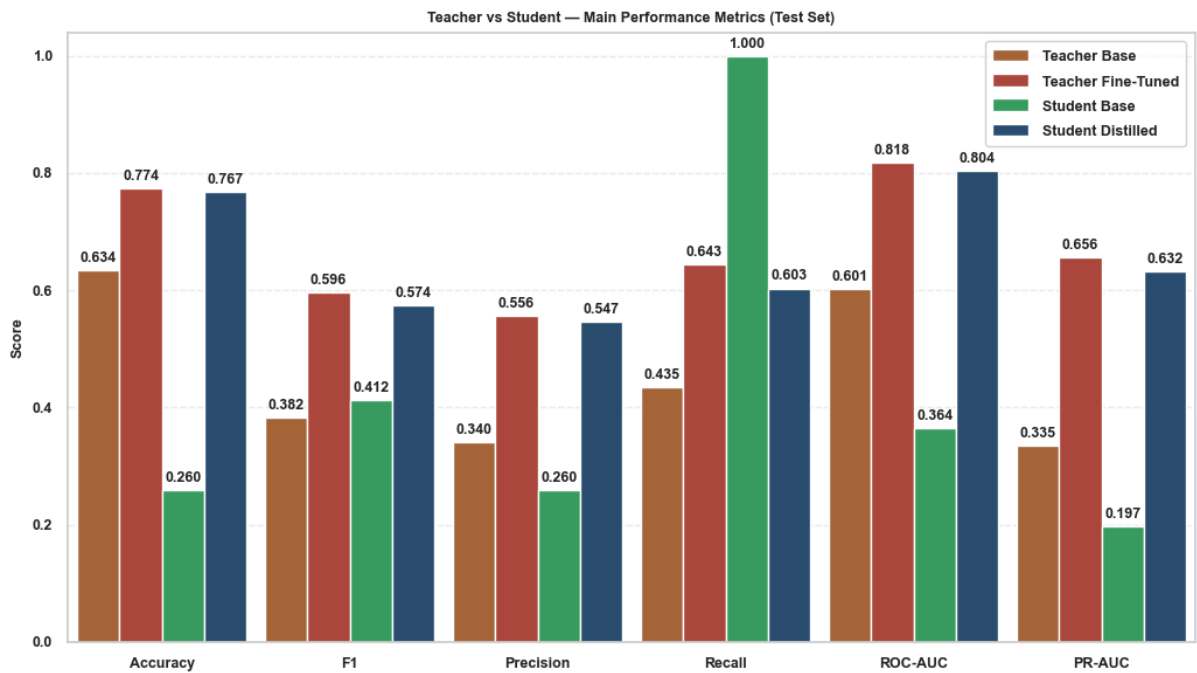
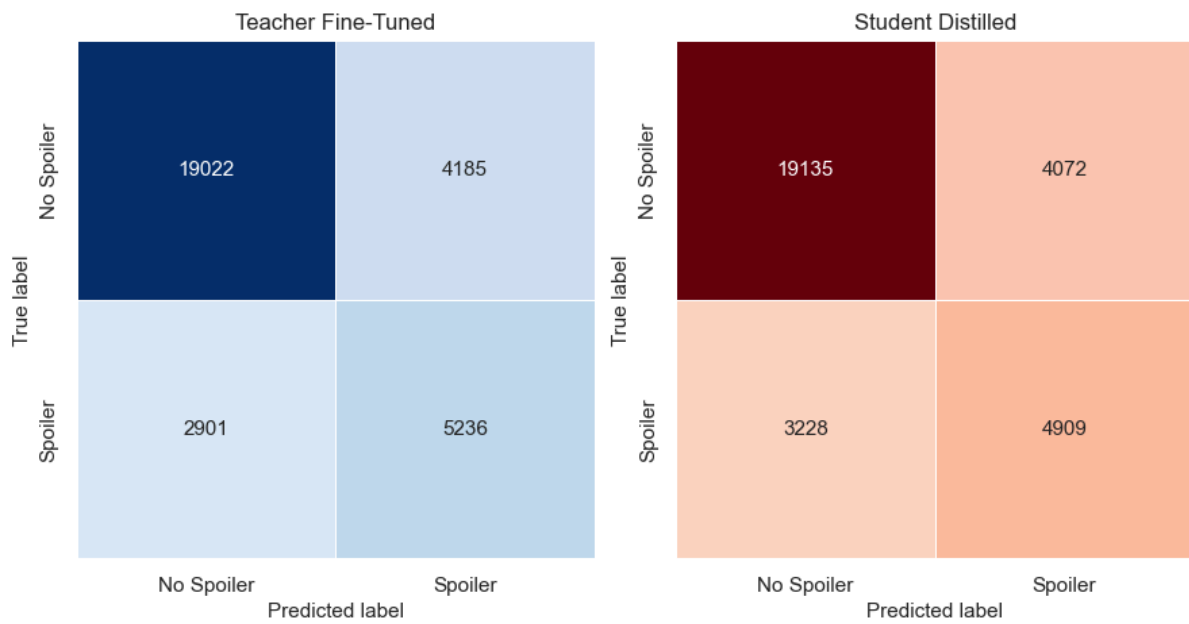
1. $\rho(x)$: È il tuo "coefficiente adattivo" (l'idea di Ji). Vale ρ_{ok} (es. 0.9) se il Teacher ha indovinato, ρ_{bad} (es. 0.2) se il Teacher ha sbagliato.
2. **Soft Loss**: È la Cross Entropy tra softmax dello Student (z_S) e softmax del Teacher (z_T) entrambi con temperatura e poi scaling di T^2 per essere già pronto nel caso in cui decidessi di aumentare la temperatura (con $T=1$ non cambia nulla). **Qui NON metti i pesi**. Ti fidi della distribuzione del Teacher così com'è (come fa TinyBERT).
3. **Hard Loss (WCE)**: È la Weighted Cross Entropy tra Student e Ground Truth (y_{true}). **Qui METTI il peso (2.8)**.

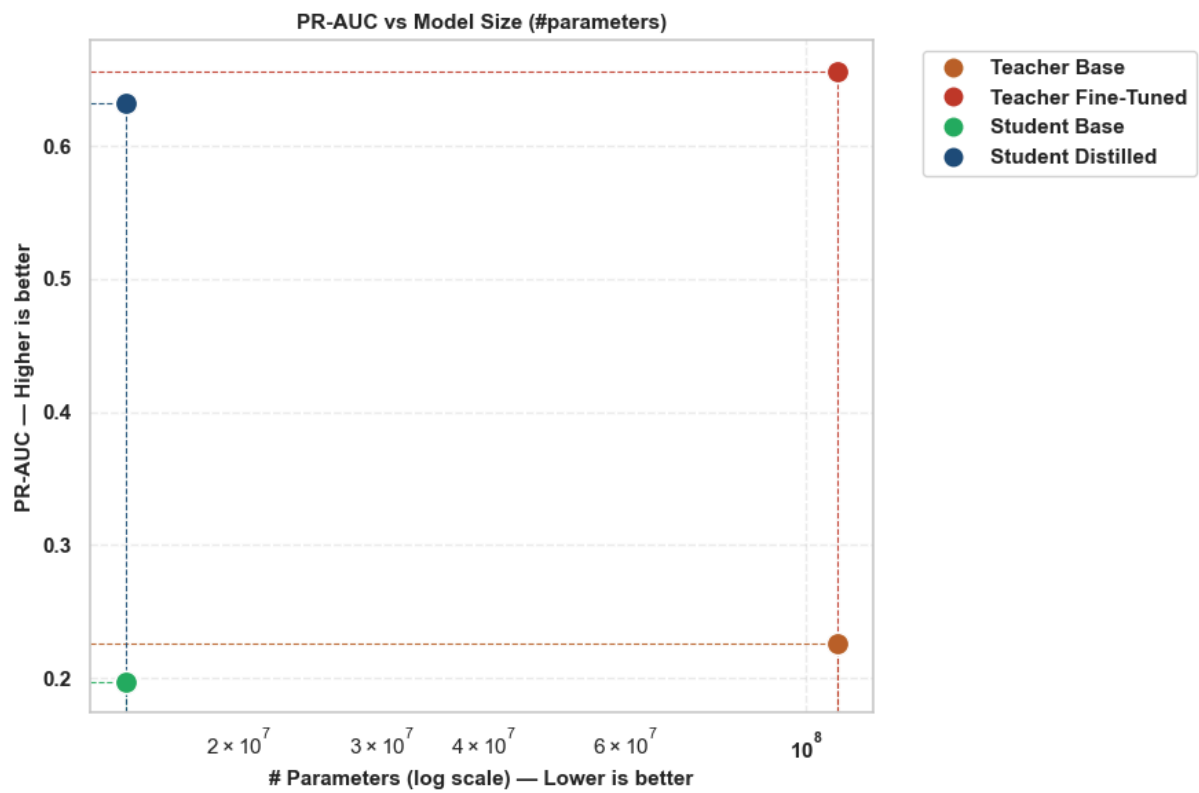
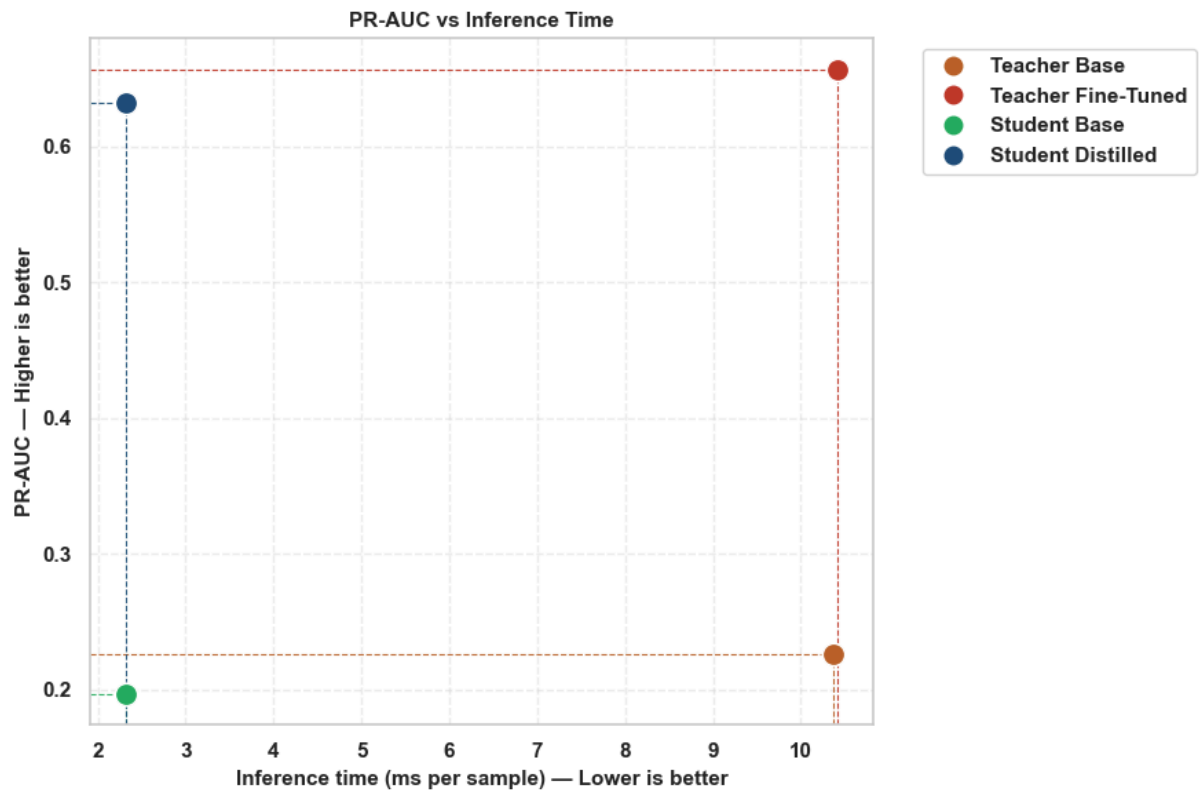
Per quanto riguarda gli altri parametri per questa fase:

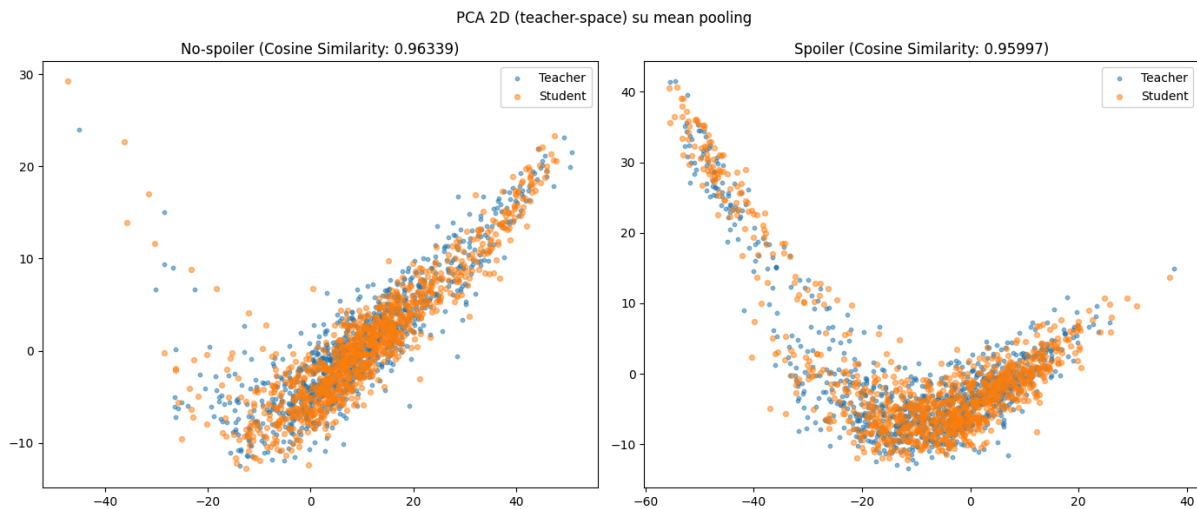
- Epochs: 3
- Batch effettivo: 32 (16 fisico + gradient accumulation)
- $LR = 2e - 5$
- Temperatura $T = 1$, se serve la aumento (i gradienti scriviamoli già come T^2)
- ρ adattivo: partiamo così e se serve modifico.
 - $\rho_{ok} = 0.9$
 - $\rho_{bad} = 0.2$

Per far fronte allo sbilanciamento del dataset, facciamo hard-loss pesata come fatto nel fine tuning (dando priorità al PR-AUC).

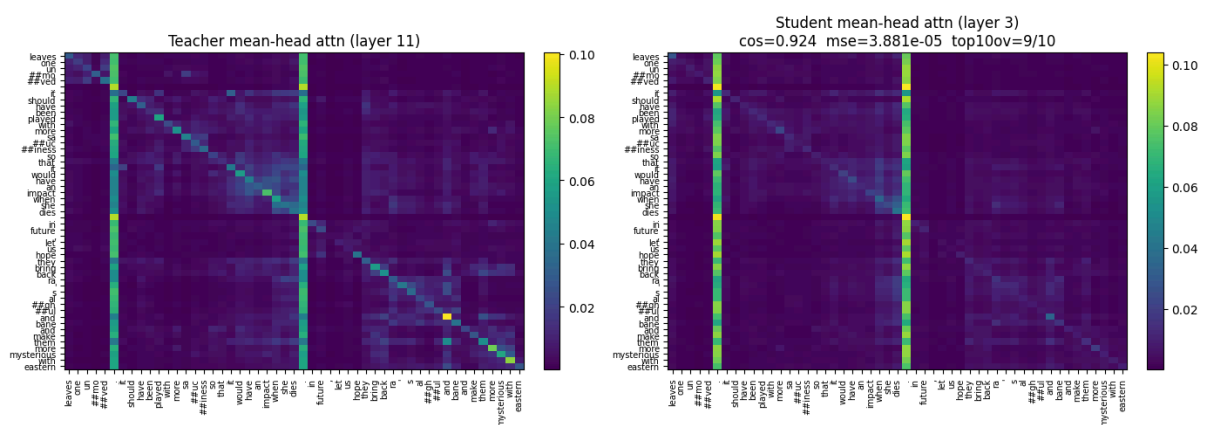
Grafici ottenuti per il momento







QUESTO È IL GRAFICO DA ESAMINARE



ANCHE QUESTO È UN GRAFICO DA ESAMINARE

Per la heatmap, la recensione data in input è stata la seguente:

"very good film. Apart from Heath ledger, the chasing sequence deserves an Oscar for stunt coordination. I thoroughly enjoyed this. The role of heroine is underplayed excellently, her sudden end leaves one unmoved. It should have been played with more sauciness so that it would have an impact when she dies. In future, let us hope they bring back Ra's Alghul and Bane and make them more mysterious with eastern mysticism. The wit, sarcasm and abrupt violence of Joker in this film acts like cathartic to the urban decay with which one is surrounded. overall, a very satisfying movie which is beginning to feel a lot more better than the graphic novels. I want to go back and see the film again and again. This is the best compliment one can pay."